

Data Structure

Level 1

Doubly Linked List Operation

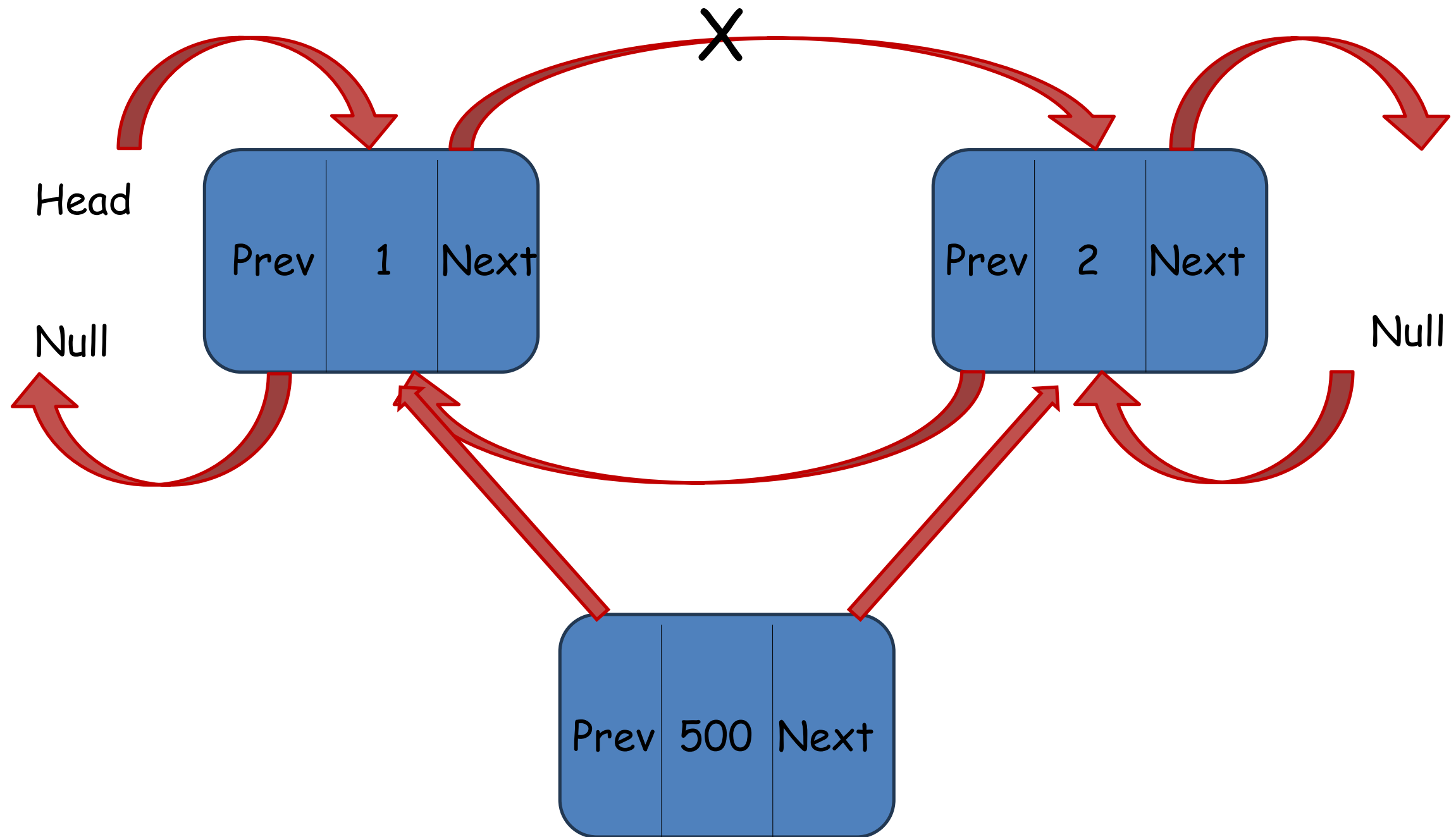


Data Structure

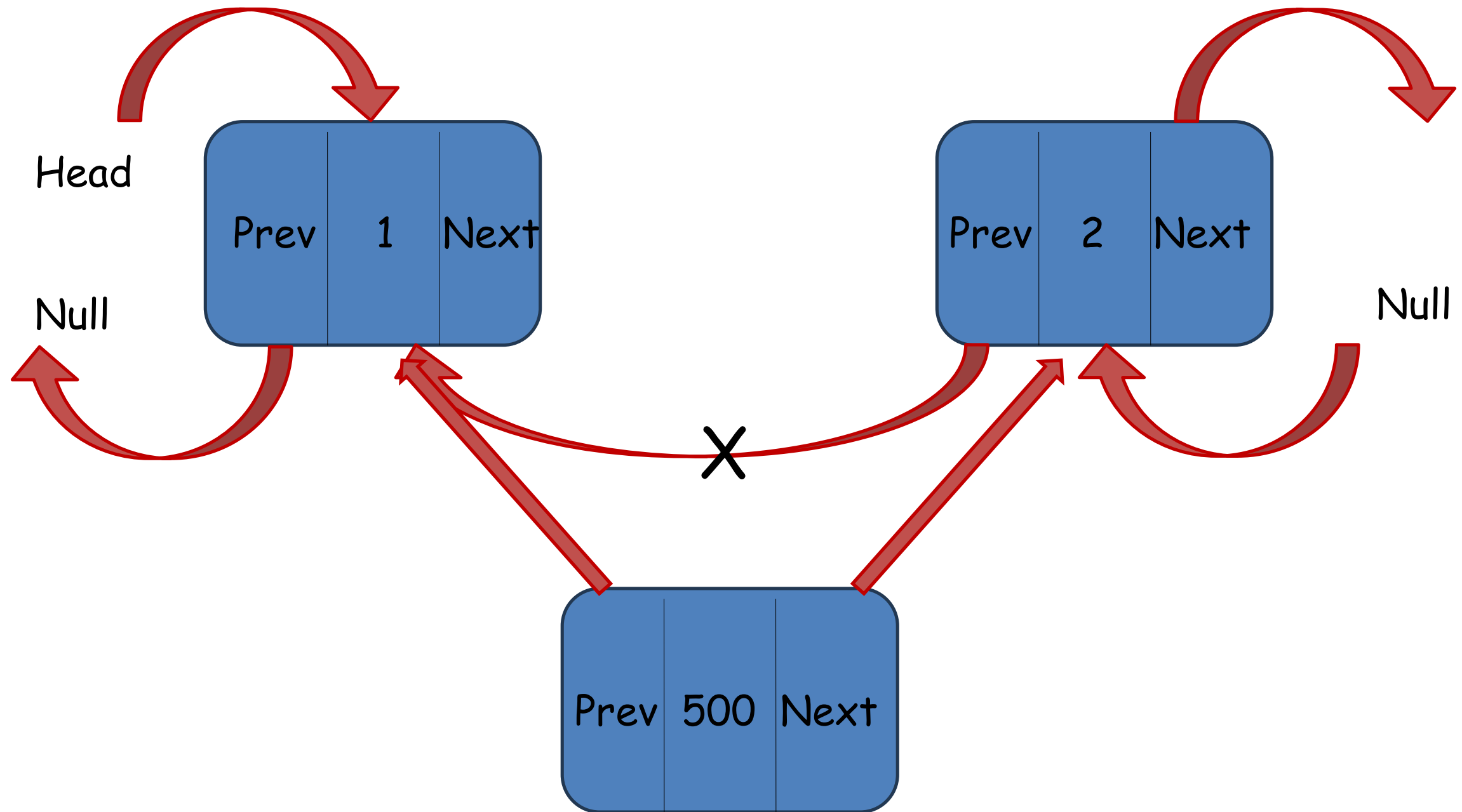
Level 1

Insert After

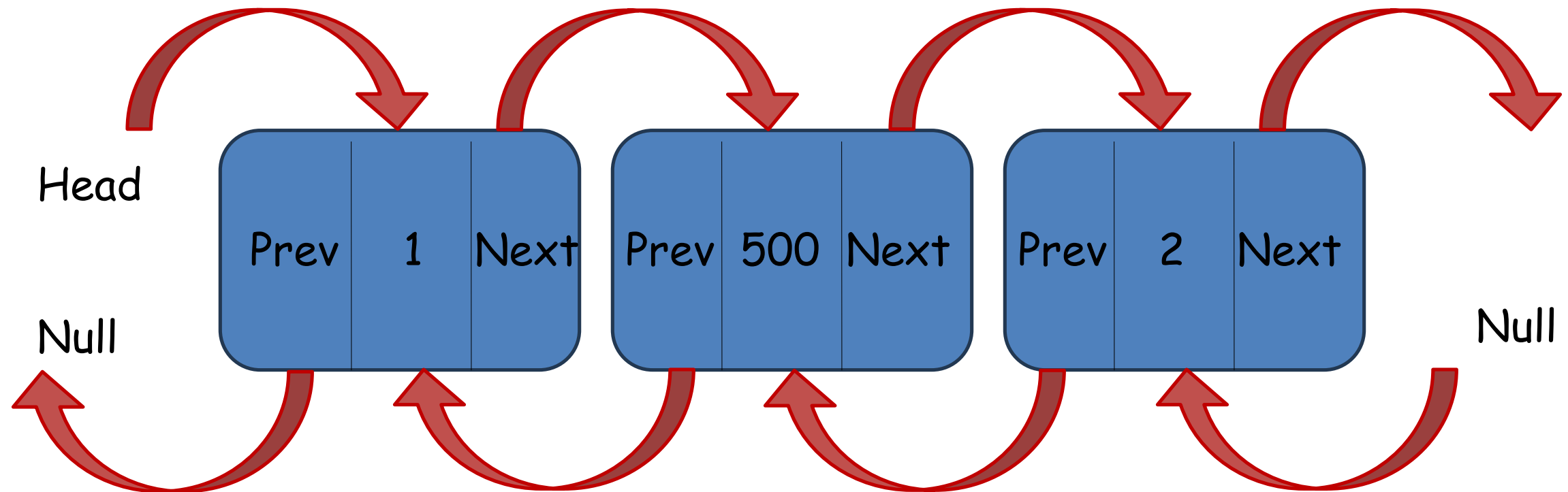
Doubley Linked List Data Structure



Doubley Linked List Data Structure



Doubley Linked List Data Structure



Definition

Adds a new node after a given node in a doubly linked list while maintaining correct next and prev links.

Explanation

- ◆ This operation inserts a node in the middle or end of the list (not at the beginning).
- ◆ It requires a reference to the target node (usually found using Find).

□ Procedures

① Find Target Node

□ Locate the node after which insertion will happen

□ Steps:

□ Traverse from head

□ Compare values

□ Return node if found

→ □ Result: Pointer to the target node

□ Procedures

2 InsertAfter Function

□ Inserts a new node after a specific node

□ Steps:

- Create a new node
- Set newNode->value
- Set newNode->next = current->next
- Set newNode->prev = current
- If next node exists → update its prev
- Link current node to new node

! Example

Before insertion:

NULL $\leftarrow$ 1 $\leftrightarrow$ 2 $\leftrightarrow$ 3 $\leftrightarrow$ 4 $\leftrightarrow$ 5 $\rightarrow$ NULL

Insert 500 after 2

After insertion:

NULL $\leftarrow$ 1 $\leftrightarrow$ 2 $\leftrightarrow$ 500 $\leftrightarrow$ 3 $\leftrightarrow$ 4 $\leftrightarrow$ 5 $\rightarrow$ NULL

Note

- ◆ Must ensure current is not NULL before inserting
- ◆ Works for:
- ◆ Middle nodes ✓
- ◆ Last node (adds at end) ✓

⊘ Warning

If `current = NULL` → program may crash ✕

⚖️❑ Comparison

❑ Insert At Beginning

- ✓❑ Time Complexity: $O(1)$
- ✓❑ No traversal needed
- ✓❑ Updates head directly

❑ Insert After (Specific Node)

- ✓❑ Time Complexity: $O(n)$ (if using Find first)
- ✓❑ Requires finding the node
- ✓❑ Updates multiple pointers

✦✦ Summary

- ✓ InsertAfter adds a node after a specific position
- ✓ Requires correct pointer updates (next & prev)
- ✓ Often combined with Find
- ✓ Works for middle and end cases

Thank you 😊